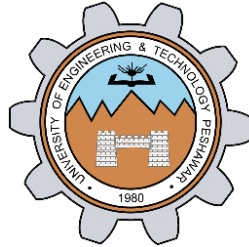


Implementation of a 3 bit up and down counter using FPGA's source clock and Clock Divider

LAB # 07



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

28th April 2024

**Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar**

Objectives:

This lab will enable students to:

- To become familiarized with behavior level modeling
- To be able to implement sequential circuits using Verilog
- To implement a 3 Bit Up and Down Counter on Spartan 6 FPGA starter kit.

Software used:

- Xilinx ISE

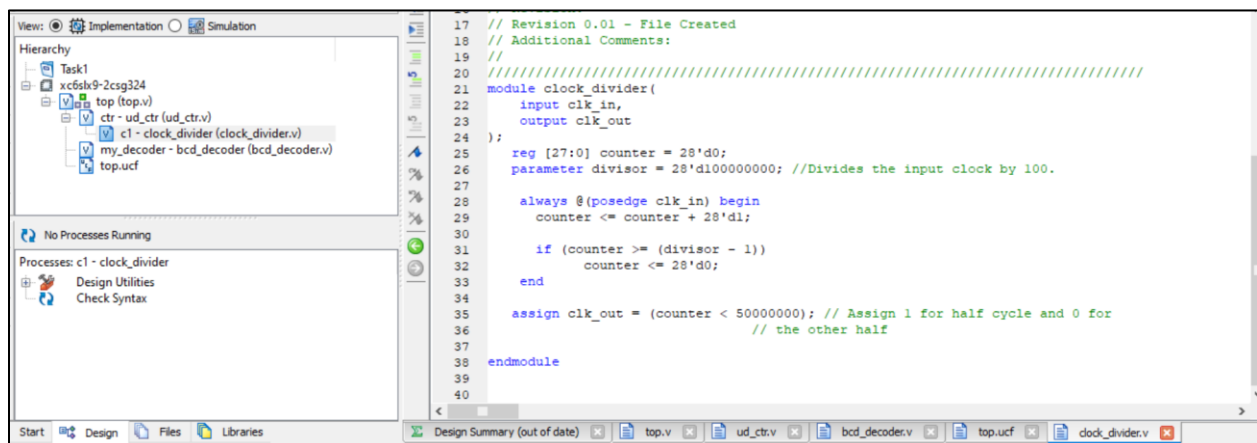
Commonly Used Modules:

The following modules are used commonly in each task. Hence, to avoid redundancy, I have pasted their working and code screenshots in this section only.

Clock Divider:

A clock divider is a circuit that takes an input clock signal and generates an output clock signal with a lower frequency. These circuits are commonly used in digital systems for various purposes, such as generating slower clock domains, creating timing signals, or driving sequential logic elements.

Code:



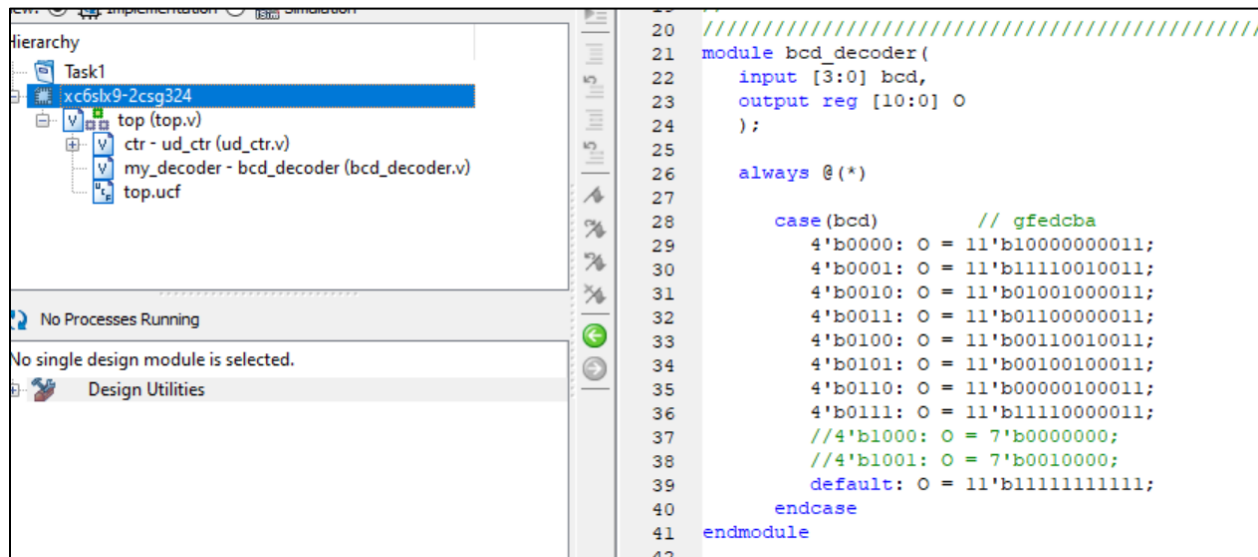
```
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////
21 module clock_divider(
22     input clk_in,
23     output clk_out
24 );
25     reg [27:0] counter = 28'd0;
26     parameter divisor = 28'd100000000; //Divides the input clock by 100.
27
28     always @(posedge clk_in) begin
29         counter <= counter + 28'd1;
30
31         if (counter >= (divisor - 1))
32             counter <= 28'd0;
33     end
34
35     assign clk_out = (counter < 50000000); // Assign 1 for half cycle and 0 for
36                                         // the other half
37
38 endmodule
39
40
```

Figure 0.1: Clock Divider Code

BCD Decoder:

BCD decoder module takes a 4-bit BCD input and produces an 11-bit output based on the specified mapping.

Code:



```
20 //////////////////////////////////////////////////
21 module bcd_decoder(
22     input [3:0] bcd,
23     output reg [10:0] O
24 );
25
26     always @(*)
27
28         case(bcd)           // gfedcba
29             4'b0000: O = 11'b100000000011;
30             4'b0001: O = 11'b11110010011;
31             4'b0010: O = 11'b01001000011;
32             4'b0011: O = 11'b01100000011;
33             4'b0100: O = 11'b00110010011;
34             4'b0101: O = 11'b00100100011;
35             4'b0110: O = 11'b00000100011;
36             4'b0111: O = 11'b11110000011;
37             //4'b1000: O = 7'b00000000;
38             //4'b1001: O = 7'b00100000;
39             default: O = 11'b11111111111;
40         endcase
41     endmodule
42
```

Figure 0.2: BCD Decoder Code

Task 1:

Implement 3-bit Up Counter using FPGA's clock source and clock divider.

Code:

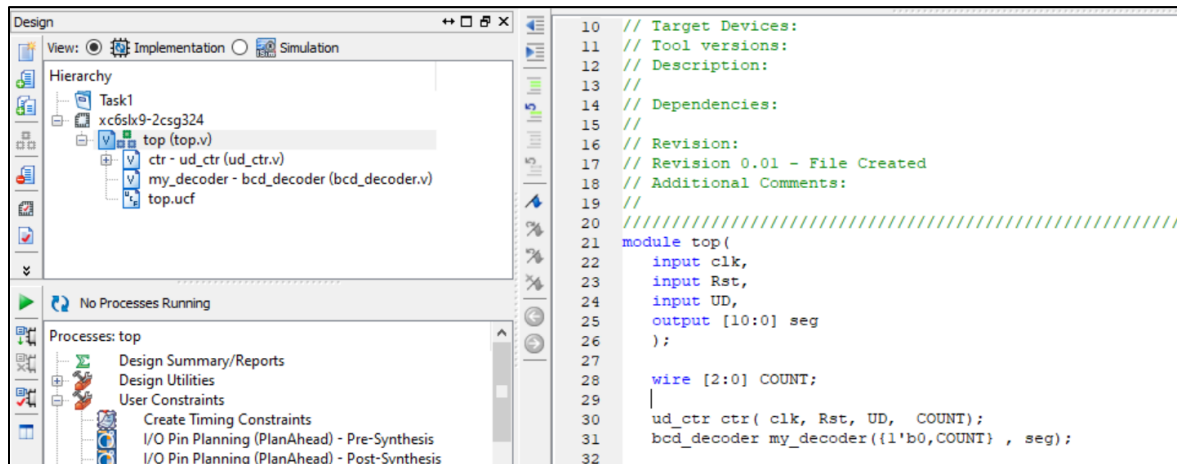


Figure 1.1 top module

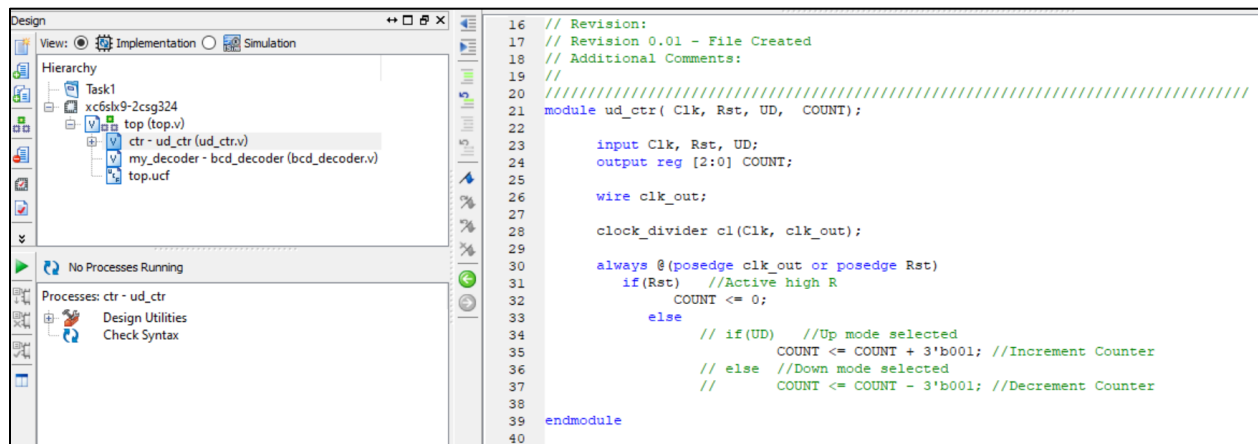


Figure 1.2: Up Counter Code

```

1  CONFIG VCCAUX = "3.3" ;
2
3  NET "clk" LOC = V10 | IOSTANDARD = LVCMOS33 | PERIOD = 100MHz ;
4
5  NET "Rst" LOC = C17 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PUL
6  # NET "UD" LOC = C18 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PU
7
8
9  NET "seg[4]" LOC = A3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
10 NET "seg[5]" LOC = B4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
11 NET "seg[6]" LOC = A4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
12 NET "seg[7]" LOC = C4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
13 NET "seg[8]" LOC = C5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
14 NET "seg[9]" LOC = D6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
15 NET "seg[10]" LOC = C6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
16 NET "seg[3]" LOC = A5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
17
18 NET "seg[2]" LOC = B3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #Enable
19 NET "seg[1]" LOC = A2 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
20 NET "seg[0]" LOC = B2 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;

```

Figure 1.3: UCF File for Up Counter

Output:

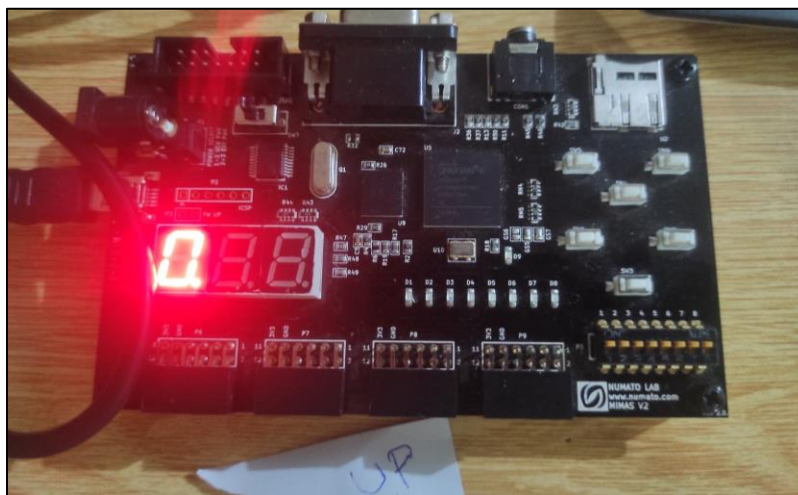


Figure 1.4: Up Counter Implementation on Spartan6 FPGA Board

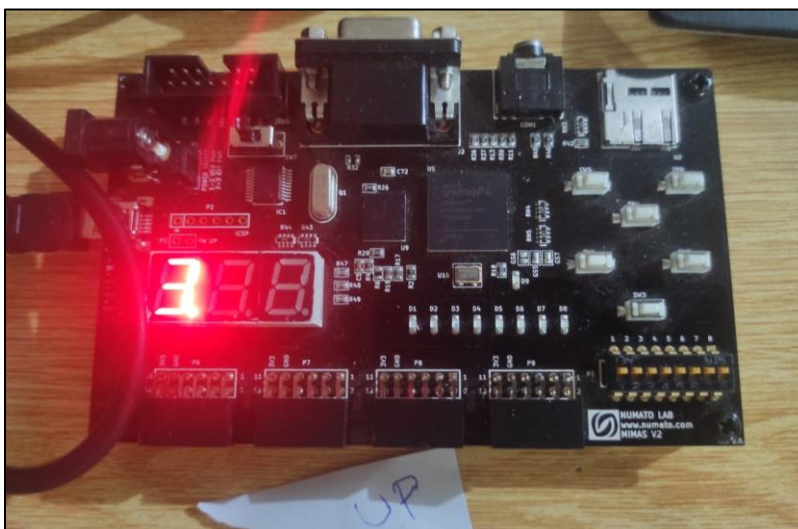


Figure 1.5: Up Counter Implementation on Spartan6 FPGA Board

Task 2:

Implement 3-bit Down Counter using FPGA's clock source and clock divider.

Code:

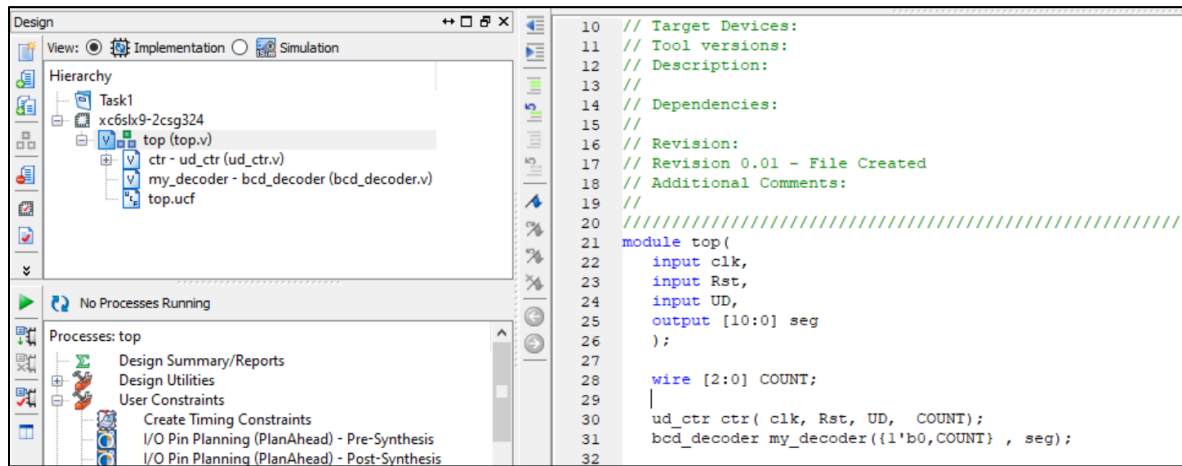


Figure 2.1 top module

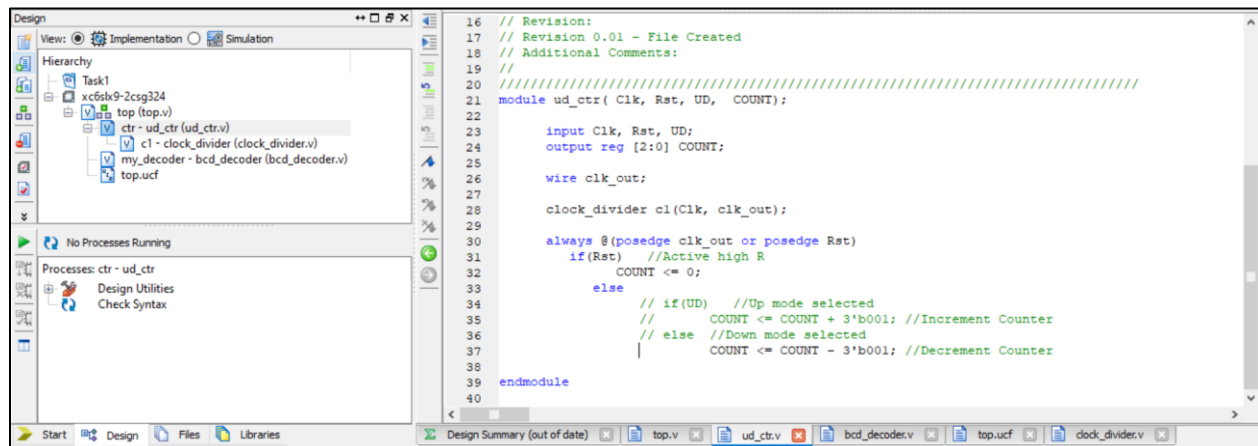


Figure 2.3: Down Counter Code


```

1  CONFIG VCCAUX = "3.3" ;
2
3  NET "clk" LOC = V10 | IOSTANDARD = LVCMOS33 | PERIOD = 100MHz ;
4
5  NET "Rst" LOC = C17 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PUL
6  # NET "UD" LOC = C18 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PU
7
8  NET "seg[4]" LOC = A3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
9  NET "seg[5]" LOC = B4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
10 NET "seg[6]" LOC = A4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
11 NET "seg[7]" LOC = C4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
12 NET "seg[8]" LOC = C5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
13 NET "seg[9]" LOC = D6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
14 NET "seg[10]" LOC = C6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
15 NET "seg[3]" LOC = A5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #
16
17 NET "seg[2]" LOC = B3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #Enable
18 NET "seg[1]" LOC = A2 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
19 NET "seg[0]" LOC = B2 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
20

```

Figure 2.3: UCF File for Down Counter

Output:

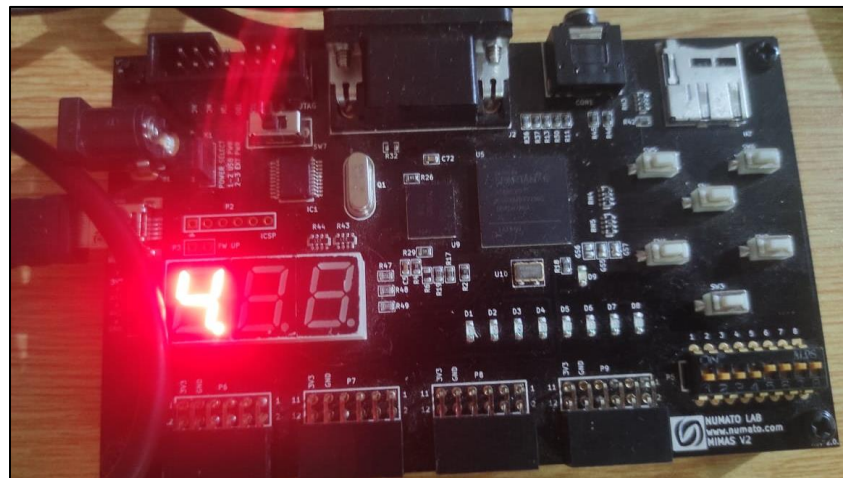


Figure 2.4: Down Counter Implementation on Spartan6 FPGA Board

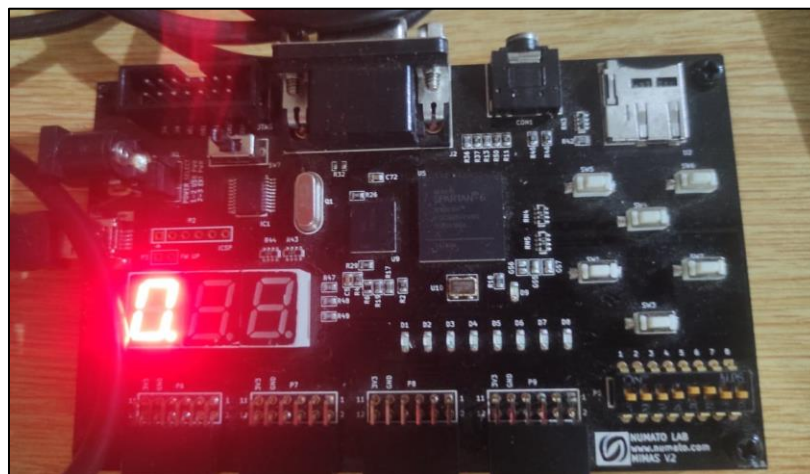


Figure 2.5: Down Counter Implementation on Spartan6 FPGA Board

Task 3:

Implement 3-bit Up and Down Counter using RST if the RST is high it should Count Up if it is low it should Count Down and display the count in the seven segment display.

Code:

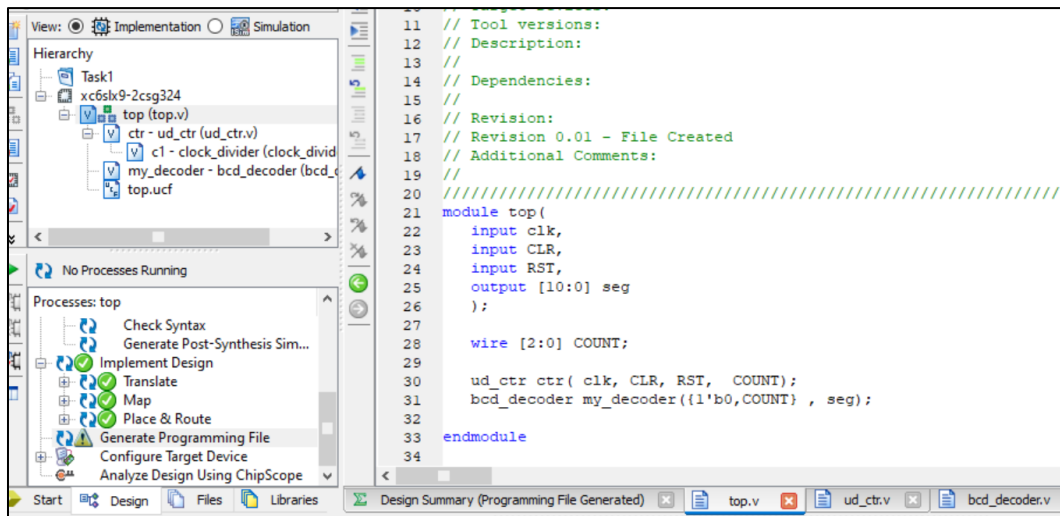


Figure 3.1 top module

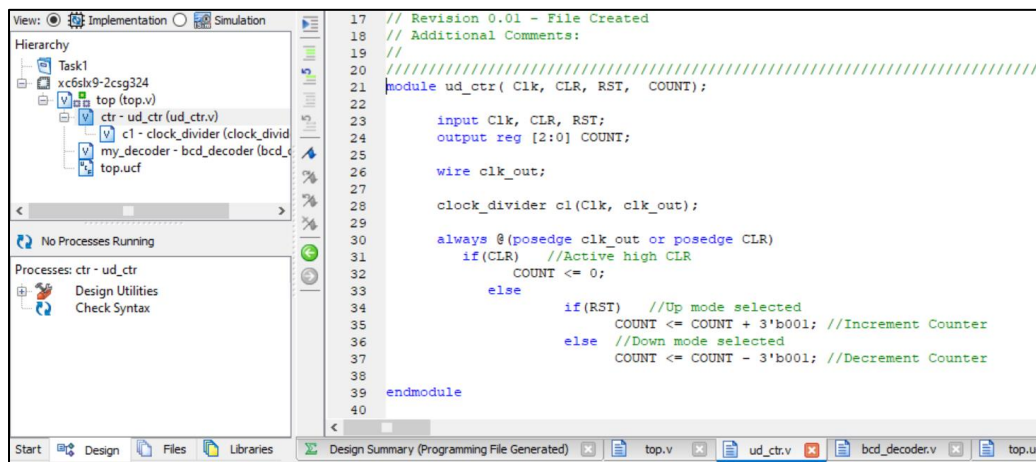


Figure 3.4: Up-Down Counter Code

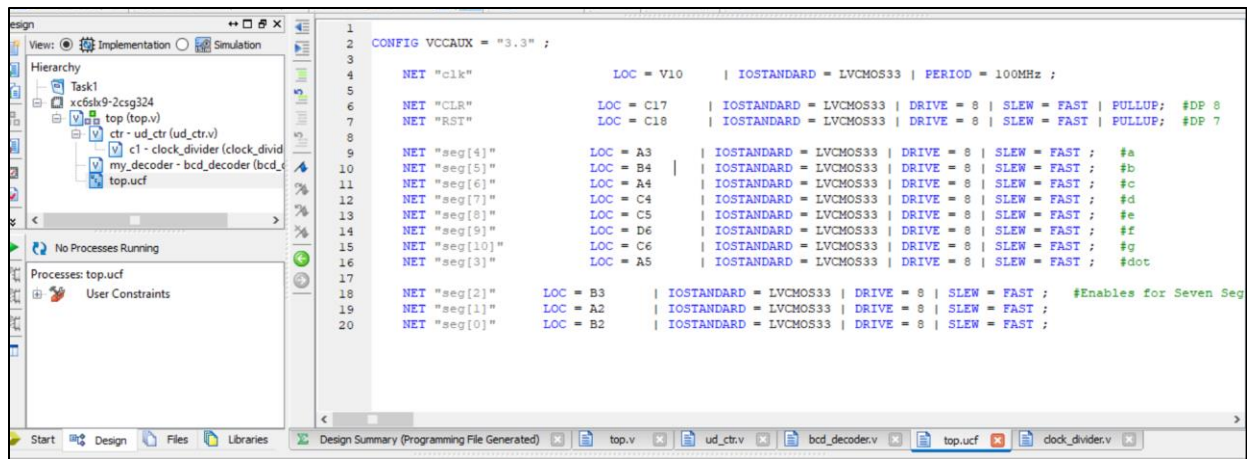


Figure 3.3: UCF File for Down Counter

Output:

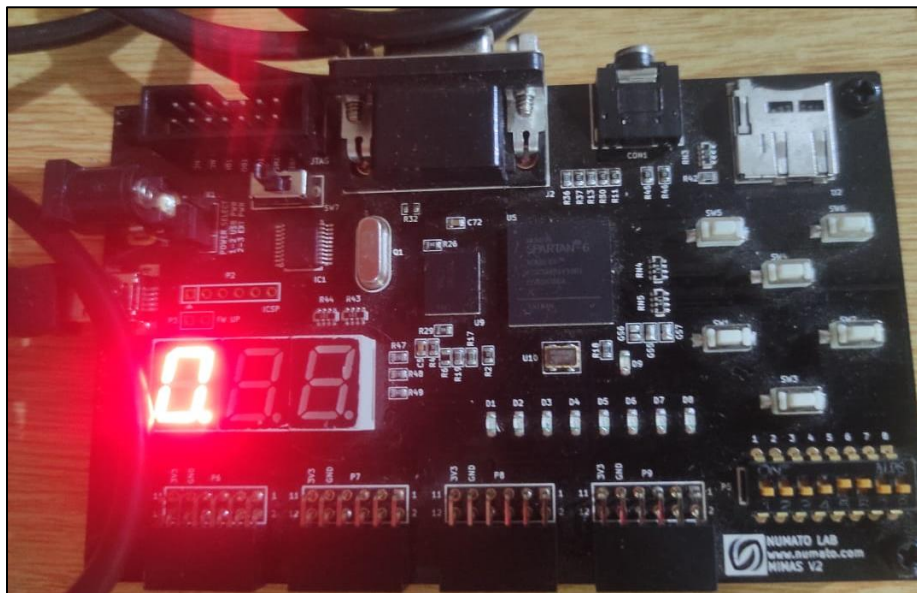


Figure 3.4: Up-Down Counter Implementation on Spartan6 FPGA Board

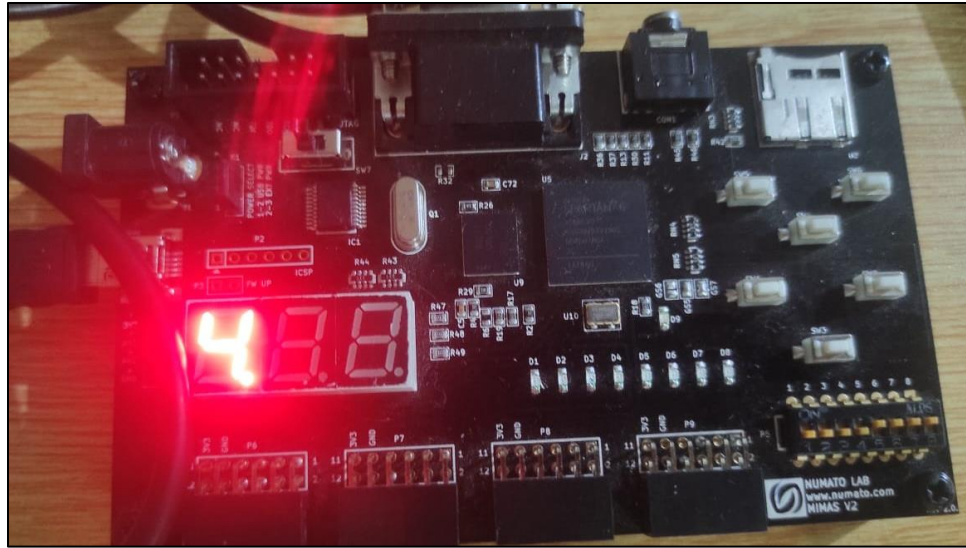


Figure 3.5: Up-Down Counter Implementation on Spartan6 FPGA Board

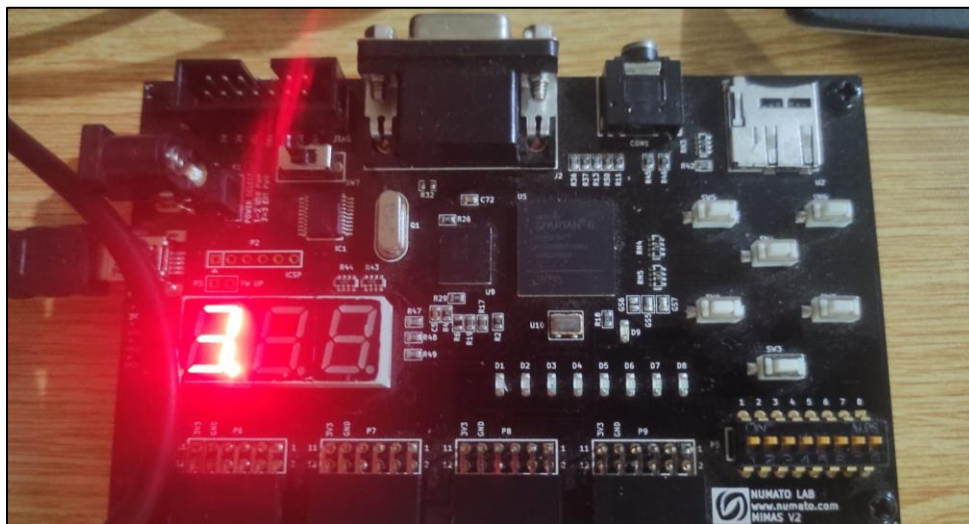


Figure 3.6: Up-Down Counter Implementation on Spartan6 FPGA Board